Relatório - Cobertura de Código e Teste de Mutação

Trabalho 2 · Verificação e Validação de Software · PUCRS · Classe testada: `CalculadoraTarifa` (cálculo do ticket de estacionamento)
Grupo / integrantes: _____

1. Procedimento

Partindo do código e dos 89 casos de teste do Trabalho 1 (projetados por particionamento em classes de equivalência e análise de valor limite), o conjunto de testes foi avaliado com duas técnicas estruturais e complementado em três etapas:

- **Etapa A - linha de base:** os 89 casos do Trabalho 1 (CT01-CT89), sem alteração.
- **Etapa B - cobertura:** análise com **JaCoCo 0.8.14** (cobertura de instruções, linhas e decisões/ramos) e, para as decisões compostas, critério **MC/DC**. Novos casos CT90-CT96.
- **Etapa C - mutação:** análise com **PIT 1.20.3** (`pitest-junit5-plugin` 1.2.2), primeiro com o conjunto de operadores padrão do plugin e depois com o grupo `STRONGER`, que usa a remoção de condicionais. Novos casos CT97-CT98.

Os casos novos seguem o mesmo padrão dos drivers do Trabalho 1 (JUnit 5, `@ParameterizedTest` + `@CsvFileSource`) e foram implementados em dois novos drivers: `TarifaCoberturaTest` (arquivos `casos_cobertura_validos.csv` e `casos_cobertura_invalidos.csv`) e `TarifaMutacaoTest` (`casos_mutacao.csv`). O código da classe testada não foi alterado. Comandos, no diretório `trab1/trabalho-estacionamento`:

```
mvn test                                                  # testes + relatório JaCoCo (target/site/jacoco)
mvn test-compile org.pitest:pitest-maven:mutationCoverage # teste de mutação (target/pit-reports)
```

2. Resumo dos resultados

Etapa	Casos	Instruções	Linhas	Ramos (decisões)	Mutantes (padrão)	Mutantes STRONGER
A - Trabalho 1 (CT01-CT89)	89	214/219 (97,7%)	45/46 (97,8%)	22/24 (91,7%)	30/32 (93,8%)	39/44 (88,6%)
B - + cobertura (CT90-CT96)	96	**219/219 (100%)**	**46/46 (100%)**	**24/24 (100%)**	30/32 (93,8%)	41/44 (93,2%)
C - + mutação (CT97-CT98)	98	**219/219 (100%)**	**46/46 (100%)**	**24/24 (100%)**	**31/32 (96,9%)**	**42/44 (95,5%)**

Os mutantes que restam vivos na etapa C (um no conjunto padrão e dois no STRONGER, todos na linha 56) são **equivalentes** (seção 4.4). Desconsiderando-os, o escore de mutação é de **100%** nos dois grupos. Os 98 casos são aprovados.

3. Análise crítica da cobertura de código

3.1 Linha de base (etapa A)

Método	Linha	Linhas	Ramos	Observação
`calcular(int, ..., boolean)`	24	3/3	-	Interface usada por todos os casos do T1.
`calcular(LocalDateTime, LocalDateTime, boolean)`	31	18/19	12/14	Linha 32 nunca executada; 2 ramos da linha 31 não exercitados.
`validarEntrada`	69	3/3	2/2	
`validarSaida`	76	4/4	4/4	
`contarPernoites`	85	6/6	2/2	

Método	Linha	Linhas	Ramos	Observação
`divisaoParaCima`	98	1/1	-	
`criarDataHora`	103	7/7	2/2	

A única lacuna está no início do método público `calcular(LocalDateTime, LocalDateTime, boolean)`:

```
31  if (entrada == null || saida == null) {                        // 2 de 4 ramos não exercitados
32      throw new IllegalArgumentException("Data/hora ... obrigatorias.");  // nunca executada
33  }
```

3.2 Análise crítica

- **Por que a lacuna existe.** Os casos do Trabalho 1 foram derivados da especificação, que descreve a entrada como dia, mês, ano, hora e minuto. Todos usam a sobrecarga com `int`, que sempre cria objetos `LocalDateTime` não nulos. A sobrecarga com `LocalDateTime` também é pública, mas só era exercitada indiretamente; por isso a validação de valores nulos nunca é verdadeira. Esse é um limite típico das técnicas funcionais: elas não enxergam caminhos que existem apenas na implementação.
- **MC/DC na linha 31.** A decisão tem duas condições, A = `entrada == null` e B = `saida == null`. A cobertura de ramos exige apenas um caso verdadeiro e um falso; o MC/DC exige mostrar que cada condição, sozinha, altera o resultado. Os pares escolhidos são: para A, CT93 (F,F) × CT90 (V,F); para B, CT93 (F,F) × CT91 (F,V). CT92 (V,V) completa a tabela-verdade. Como o Java avalia `||` em curto-circuito, B não chega a ser avaliada em CT90 e CT92.
- **MC/DC na linha 78** (`validarSaida`). A decisão `!hora.isBefore(FECHAMENTO) && hora.isBefore(ABERTURA)` já satisfazia o critério com os casos do T1, sem casos novos: para A = hora ≥ 02:00, o par CT71 (05:00; V,V → exceção) × CT22 (01:59; F,V → válida); para B = hora < 08:00, o par CT71 × CT38 (08:00; V,F → válida). As demais decisões do código têm uma única condição, e a cobertura de ramos de 100% já equivale ao MC/DC nelas.
- **Cobertura de 100% não basta.** As linhas 35 e 36 (`truncatedTo(ChronoUnit.MINUTES)`) já eram executadas por todos os casos, mas nenhum deles verificava o efeito do truncamento: com a interface `int` os segundos são sempre zero. Se as duas linhas fossem apagadas, os 89 casos continuariam passando. Como a sobrecarga com `LocalDateTime` aceita segundos e nanossegundos, foram criados casos em que o resultado muda conforme o truncamento seja feito ou não (CT94-CT96). A eficácia deles foi confirmada removendo as chamadas: CT94 e CT96 falham na asserção e CT95 lança exceção.

3.3 Novos casos de teste (cobertura)

ID	Critério / objetivo	Entrada	Saída	VIP	t (min)	Resultado esperado
CT90	MC/DC linha 31: A=V, B=F (ramo verdadeiro, linha 32)	null	10/03/2025 10:30	Não	-	IllegalArgumentException
CT91	MC/DC linha 31: A=F, B=V	10/03/2025 10:00	null	Não	-	IllegalArgumentException
CT92	MC/DC linha 31: A=V, B=V	null	null	Não	-	IllegalArgumentException
CT93	MC/DC linha 31: A=F, B=F (chamada direta da sobrecarga)	10/03/2025 10:00	10/03/2025 10:21	Não	21	R$ 15,00
CT94	Truncamento: sem truncar, a permanência seria de 20 min (cortesia)	10/03/2025 10:00:30	10/03/2025 10:21:10	Não	21	R$ 15,00
CT95	Truncamento: mesmo minuto, segundos da saída menores que os da entrada (sem truncar, a saída seria anterior à entrada)	10/03/2025 10:00:30	10/03/2025 10:00:10	Não	0	R$ 0,00
CT96	Truncamento + VIP na fronteira 60/61 min (sem truncar, t seria 60)	10/03/2025 10:00:30.5	10/03/2025 11:01:10	Sim	61	R$ 10,00

Com esses casos, a cobertura de instruções, linhas e ramos passa a 100% (etapa B na seção 2).

4. Análise crítica do teste de mutação

4.1 Configuração

O PIT foi aplicado à classe `CalculadoraTarifa` com todos os drivers de teste. Foram usados dois grupos de operadores:

- **Padrão do plugin** (32 mutantes, usado quando o `pom.xml` não define operadores): limite condicional, negação de condicional, operadores aritméticos, incremento, valores de retorno e remoção de chamadas `void`.
- `STRONGER` (44 mutantes), adotado como configuração final no `pom.xml`: troca a negação de condicionais pela **remoção de condicionais**, que substitui cada desvio por `true` e por `false`, gerando mais mutantes por decisão. Distribuição dos 44 mutantes por operador:

Operador	Mutantes
Remoção de condicional (igualdade → false)	8
Remoção de condicional (igualdade → true)	8
Operador aritmético	8
Retorno primitivo	4
Limite condicional	4
Remoção de condicional (comparação → false)	4
Remoção de condicional (comparação → true)	4
Remoção de chamada void	2
Incremento	1
Retorno nulo	1

Mutantes encerrados por *timeout* são considerados detectados: na linha 88, trocar a condição do laço `while (saida.isAfter(marco))` por `true` gera um laço infinito, que o PIT interrompe.

4.2 Linha de base (etapa A): mutantes sobreviventes

Linha	Código original	Mutação	Grupo	Diagnóstico
31	`entrada == null`	condição substituída por `false` (nunca rejeita entrada nula)	STRONGER	Falta de caso com entrada nula: é a mesma lacuna apontada pelo JaCoCo.
31	`saida == null`	condição removida (nunca rejeita saída nula)	STRONGER	Falta de caso com saída nula.
103	`ano < 1`	`ano <= 1` (limite condicional)	ambos	Valor limite incompleto: o T1 testou o ano 0 (inválido, CT85 e CT89), mas não o ano 1, primeiro valor válido.
56	`minutos <= MINUTOS_POR_HORA`	`minutos < MINUTOS_POR_HORA` (limite condicional)	ambos	Equivalente (seção 4.4).
56	`minutos <= MINUTOS_POR_HORA`	condição substituída por `false`	STRONGER	Equivalente (seção 4.4).

4.3 Análise crítica

- **O conjunto padrão escondia a fragilidade da linha 31.** Com ele, os mutantes da linha 31 são do tipo negação (`==` → `!=`). Eles fazem todo caso válido lançar exceção, então qualquer teste os mata, mesmo sem nenhum caso com valor nulo. Só a remoção de condicionais (`STRONGER`) revela que a validação de nulos poderia ser apagada sem que os testes do T1 percebessem. Por isso o grupo `STRONGER` foi adotado.
- **Cobertura e mutação se complementam.** Os dois mutantes da linha 31 foram mortos pelos casos criados na etapa de cobertura (CT90-CT92), sem casos adicionais. Já o mutante da linha 103 sobrevive com 100% de cobertura: a linha é executada (pelo ano 0 e pelos anos válidos), mas nenhum caso distingue `< 1` de `<= 1`. A cobertura mede o que é executado; a mutação mede o que é **verificado**.

- **O mutante da linha 103 confirma a importância dos valores limite.** Ele corresponde a um defeito plausível de *off-by-one* que rejeitaria o ano 1, data válida para `LocalDateTime`. O par de valores limite da fronteira do ano ficou incompleto no T1, e os casos CT97-CT98 o completam.
- **Relação com os defeitos reais do Trabalho 1.** Dos 7 defeitos da versão 1, só o D1 (`<` no lugar de `<=` na cortesia, linha 47) corresponde diretamente a um operador do PIT (limite condicional), e o mutante correspondente é morto por CT04. D3, D5 e D7 também são erros de fronteira, mas em chamadas a `isAfter`/`isBefore` do `java.time`, que o PIT não altera. D2 (fórmula), D4 (`return` antecipado) e D6 (`Math.min`) também não pertencem aos operadores disponíveis. Assim, um escore de mutação alto não garante a detecção de todos os tipos de defeito: os casos de valor limite do T1 continuam necessários.

4.4 Mutantes equivalentes (linha 56)

```
54  if (pernoites > 0) {
55      valor = pernoites * TARIFA_PERNOITE;
56  } else if (minutos <= MINUTOS_POR_HORA) {     // mutantes sobreviventes
57      valor = TARIFA_FIXA;
58  } else {
59      valor = TARIFA_FIXA + ADICIONAL_POR_HORA * divisaoParaCima(minutos - 60, 60);
    }
    // divisaoParaCima(d, 60) = (d + 59) / 60   (divisão inteira)
```

A linha 56 só é alcançada com `minutos > 20` (cortesia tratada antes) e sem pernoite. Uma pernoite exige ao menos 482 minutos (entrada 23:59 e saída 08:01), portanto todo caso com t ≤ 60 chega à linha 56.

- **`<=` → `<`:** a diferença ocorre só em t = 60. O original cobra R$ 15,00 pela linha 57; o mutante segue para a linha 59 e cobra 15 + 5 × ((0 + 59) / 60) = 15 + 5 × 0 = **R$ 15,00**.
- **condição → `false`:** para 21 ≤ t ≤ 60, o mutante segue sempre para a linha 59, com d = t − 60 entre −39 e 0, e (d + 59) / 60 tem numerador entre 20 e 59, logo vale 0. Valor: **R$ 15,00**, igual ao original.

Nenhuma entrada distingue os mutantes do original, e por isso nenhum caso de teste pode matá-los. Eles mostram que o ramo `else if` é **redundante**: a fórmula das horas adicionais já devolve a tarifa fixa para qualquer permanência de até 60 minutos. Uma simplificação possível é remover o ramo; como o objetivo é avaliar a classe entregue, o código não foi alterado.

4.5 Novos casos de teste (mutação)

ID	Mutante a matar / técnica	Entrada	Saída	VIP	t (min)	Resultado esperado
CT97	Linha 103, ano `< 1` → ano `<= 1`; VL: ano 1 (primeiro ano válido)	01/01/0001 10:00	01/01/0001 10:30	Não	30	R$ 15,00
CT98	Linha 103, mesmo mutante; VL: virada do ano 1 para o ano 2 com pernoite	31/12/0001 23:00	01/01/0002 09:00	Não	600	R$ 50,00

Para os casos CT90-CT92, criados na etapa de cobertura, também está registrado quais mutantes da linha 31 eles matam (Apêndice A).

4.6 Resultado final

Com os 98 casos, o PIT (`STRONGER`) gera 44 mutantes: 41 são mortos por falha de teste, 1 por *timeout* e 2 sobrevivem, ambos equivalentes. O escore de mutação é de **95,5%** (42/44); desconsiderando os equivalentes, de **100%** (42/42).

5. Conclusão

Com os 9 casos novos (CT90-CT98), o conjunto passa a ter 98 casos, 100% de cobertura de instruções, linhas e ramos, MC/DC nas duas decisões compostas e todos os mutantes não equivalentes mortos. Com base nesses critérios, a classe `CalculadoraTarifa` entregue (versão corrigida) é **aceita**. A análise também mostrou que:

- cobertura de 100% não garante testes eficazes (truncamento e ano 1);
- o conjunto padrão de operadores do PIT pode esconder fragilidades que a remoção de condicionais revela;

- mutantes equivalentes apontam código redundante;
- a análise de mutação não substitui o projeto por valor limite, pois a maioria dos defeitos reais do Trabalho 1 não corresponde a operadores do PIT.

Apêndice A - Mutantes gerados (PIT, grupo STRONGER, 98 casos)

A coluna "Morto por" indica o primeiro caso de teste que detectou o mutante.

Mutante	Linha	Método	Operador	Situação	Morto por
M01	26	`calcular`	Retorno primitivo	Morto	CT07
M02	31	`calcular`	Remoção de condicional (igualdade → false)	Morto	CT03
M03	31	`calcular`	Remoção de condicional (igualdade → false)	Morto	CT91
M04	31	`calcular`	Remoção de condicional (igualdade → true)	Morto	CT90
M05	31	`calcular`	Remoção de condicional (igualdade → true)	Morto	CT03
M06	38	`calcular`	Remoção de chamada void	Morto	CT63
M07	39	`calcular`	Remoção de chamada void	Morto	CT72
M08	41	`calcular`	Remoção de condicional (igualdade → false)	Morto	CT76
M09	41	`calcular`	Remoção de condicional (igualdade → true)	Morto	CT03
M10	47	`calcular`	Limite condicional	Morto	CT04
M11	47	`calcular`	Remoção de condicional (comparação → false)	Morto	CT03
M12	47	`calcular`	Remoção de condicional (comparação → true)	Morto	CT07
M13	54	`calcular`	Limite condicional	Morto	CT07
M14	54	`calcular`	Remoção de condicional (comparação → false)	Morto	CT43
M15	54	`calcular`	Remoção de condicional (comparação → true)	Morto	CT07
M16	55	`calcular`	Operador aritmético	Morto	CT42
M17	56	`calcular`	Limite condicional	Sobrevivente	equivalente
M18	56	`calcular`	Remoção de condicional (comparação → false)	Sobrevivente	equivalente
M19	56	`calcular`	Remoção de condicional (comparação → true)	Morto	CT22
M20	59	`calcular`	Operador aritmético	Morto	CT22
M21	61	`calcular`	Operador aritmético	Morto	CT24
M22	61	`calcular`	Operador aritmético	Morto	CT22
M23	64	`calcular`	Operador aritmético	Morto	CT32
M24	64	`calcular`	Remoção de condicional (igualdade → false)	Morto	CT32
M25	64	`calcular`	Remoção de condicional (igualdade → true)	Morto	CT07
M26	64	`calcular`	Retorno primitivo	Morto	CT07
M27	69	`validarEntrada`	Remoção de condicional (igualdade → false)	Morto	CT63
M28	69	`validarEntrada`	Remoção de condicional (igualdade → true)	Morto	CT03
M29	78	`validarSaida`	Remoção de condicional (igualdade → false)	Morto	CT72
M30	78	`validarSaida`	Remoção de condicional (igualdade → false)	Morto	CT72
M31	78	`validarSaida`	Remoção de condicional (igualdade → true)	Morto	CT20
M32	78	`validarSaida`	Remoção de condicional (igualdade → true)	Morto	CT03
M33	88	`contarPernoites`	Remoção de condicional (igualdade → false)	Morto	CT43
M34	88	`contarPernoites`	Remoção de condicional (igualdade → true)	Morto (timeout)	timeout: laço infinito
M35	89	`contarPernoites`	Incremento	Morto	CT42

Mutante	Linha	Método	Operador	Situação	Morto por
M36	93	`contarPernoites`	Retorno primitivo	Morto	CT43
M37	98	`divisaoParaCima`	Operador aritmético	Morto	CT22
M38	98	`divisaoParaCima`	Operador aritmético	Morto	CT22
M39	98	`divisaoParaCima`	Operador aritmético	Morto	CT22
M40	98	`divisaoParaCima`	Retorno primitivo	Morto	CT22
M41	103	`criarDataHora`	Limite condicional	Morto	CT98
M42	103	`criarDataHora`	Remoção de condicional (comparação → false)	Morto	CT85
M43	103	`criarDataHora`	Remoção de condicional (comparação → true)	Morto	CT03
M44	108	`criarDataHora`	Retorno nulo	Morto	CT03

Apêndice B - Arquivos entregues

Arquivo	Conteúdo
`pom.xml`	Configuração do JaCoCo (`prepare-agent` e `report` na fase `test`) e do PIT (grupo `STRONGER`, relatórios HTML e XML).
`src/main/java/.../CalculadoraTarifa.java`	Classe testada (versão corrigida do T1, sem alterações).
`TarifaPermanenciaTest`, `TarifaPernoiteTest`, `TarifaEntradasInvalidasTest`	Drivers do T1 (CT01-CT89).
`TarifaCoberturaTest` + `casos_cobertura_*.csv`	Casos da análise de cobertura (CT90-CT96).
`TarifaMutacaoTest` + `casos_mutacao.csv`	Casos da análise de mutação (CT97-CT98).